

Trabajo Práctico Integrador - Programación 1

Materia: Programación 1

Carrera: Tecnicatura en Programación a Distancia
(UTN)

Título del Proyecto: Gestión de Datos de Países en
Python: filtros, ordenamientos y estadísticas

Alumno: Daniela Carolina Fernanda Burgos

Fecha: 29 de junio de 2026

Índice

Marco Teórico	2
Decisiones Técnicas y Arquitectura	4
Dificultades y Conclusiones:.....	8
Bibliografía Bibliografía / Webgrafía	9
Anexo	9
Link al repositorio del proyecto.....	9
Link al video explicativo del proyecto	9

Marco Teórico

El presente trabajo práctico integrador requiere aplicar diversos conceptos de programación en Python para dar cumplimiento a la consigna compartida sobre la gestión de datos de países.

Serán compartidas a continuación las bases teóricas necesarias: estructuras de datos como listas y diccionarios, la modularización mediante funciones, el uso de condicionales y técnicas de ordenamiento y estadísticas y el uso de Pandas para la lectura del CSV.

1. Listas

Tal como explica la documentación de Python, una lista es una estructura de datos mutable y ordenada, definidas por corchetes “[]” que permite almacenar colecciones de elementos, separados por comas (.). En este proyecto, las listas se utilizan como la estructura principal para almacenar los países cargados y acceder a ellos por índices. Esto permite establecer relaciones sobre ellos de manera sencilla para realizar comparaciones para búsquedas, filtros y ordenamientos.

2. Diccionarios

Un diccionario en Python es una colección mutable y desordenada de pares "clave-valor", definidas por llaves “{ }”. Son ideales para representar entidades estructuradas. En el sistema cada país está modelado como un diccionario, donde las claves son los atributos solicitados en la consigna (nombre, población, superficie, continente) y los valores contienen la información específica de dicho país.

3. Funciones

Las funciones permiten modularizar el código, asegurando que sea más ordenado y de fácil mantenimiento. Permite separar el código en bloques reutilizables y organizados, delegando cada acción que debe hacer el código a una función específica, llamada en el bloque principal. Sus elementos principales son la función def() y return().

4. Condicionales

Las estructuras condicionales (if, elif, else) permiten al código tomar una acción u otra, basándose en el cumplimiento de ciertas condiciones lógicas. En este desarrollo, son esenciales para construir el menú interactivo (ejecutando una función u otra según la entrada del usuario) y para aplicar filtros de búsqueda (por ejemplo, evaluar si la población de un país está dentro de un rango solicitado).

5. Ordenamientos

Python provee funciones integradas como `sorted()` que permiten ordenar listas o diccionarios de datos. En el TPI, esta herramienta se aplica para mostrar la lista de diccionarios (países) ordenados según criterios específicos elegidos por el usuario (alfabético por nombre, numérico ascendente/descendente por población o superficie).

6. Estadísticas Básicas

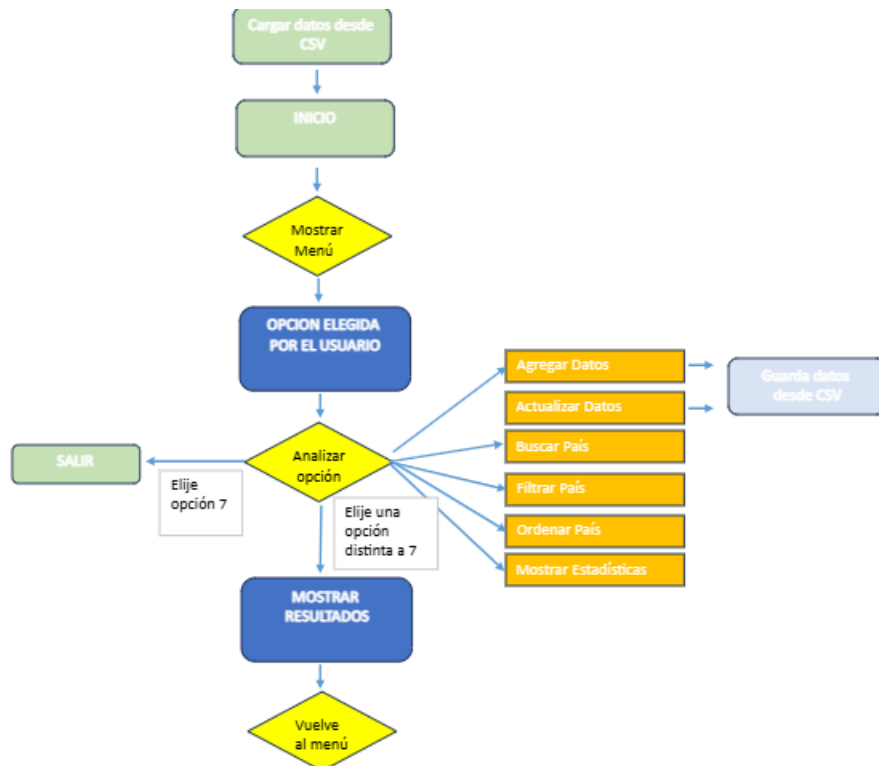
La estadística básica en el código implica que se aplican operaciones como `max()`, `min()` y promedios. El sistema implementa acumuladores y comparadores clásicos para calcular el promedio total (suma de valores dividida por la cantidad de elementos), así como algoritmos de búsqueda de máximos y mínimos para determinar qué país tiene la mayor o menor población.

7. Archivos CSV

Un archivo CSV (Valores Separados por Comas) es un formato de texto utilizado para representar datos estructuras y separados por comas. Para garantizar la persistencia de los datos, el proyecto emplea el módulo estándar `csv` de Python. Esto permite que la información de los países no se pierda al cerrar el programa, logrando importar los registros al iniciar y exportar nuevos datos. En el proyecto lo usamos para cargar el dataset de los países.

Decisiones Técnicas y Arquitectura

Diagrama de flujo de operaciones principales:



El diagrama de flujo representa la lógica general del programa de gestión de países. El sistema comienza con un dataset previamente cargado, posteriormente el usuario da inicio al programa, donde se muestra el menú principal con las distintas opciones disponibles para el usuario. A continuación, el usuario selecciona una opción y el programa analiza la decisión mediante estructuras condicionales.

Cada opción del menú corresponde a una función específica: Agregar datos y Actualizar, implican que la información se guarde en el csv de datos, Buscar país, Filtrar países, Ordenar datos y Mostrar estadísticas. Una vez ejecutada la acción, el sistema muestra los resultados correspondientes en pantalla.

Posteriormente, vuelve a mostrarse el menú. Si se selecciona cualquier opción del 1 al 6, el flujo regresa al menú principal y el ciclo se repite. En caso de elegir la opción 7, el programa finaliza en el bloque de Salir.

Este esquema permite visualizar de manera clara y ordenada la interacción entre el usuario y el sistema, mostrando cómo cada decisión conduce a un proceso específico y cómo se garantiza la repetición del menú hasta que el usuario decida finalizar la ejecución.

Capturas de pantalla de la ejecución de ejemplos:

Ejemplo 1: tenemos el csv “Datos_paises.csv” previamente cargado. Ejecutamos el programa. Escogemos la opción 1. Agregar país. Ingresamos los datos solicitados. Guarda el dato en el csv:

EXPLORER

PROGRAMACIÓN TPI

Datos_paises.csv

TPI entregable.py

TPI.py

TPI entregable.py

Datos_paises.csv

```
21 return paises
22
23 def guardar_paises(paises):
24     with open("Datos_paises.csv", "w", newline="", encoding="utf-8") as archivo:
25         campos = ["nombre", "poblacion", "superficie", "continente"]
26         escritor = csv.DictWriter(archivo, fieldnames=campos)
27         escritor.writeheader()
28         escritor.writerows(paises)
29
30 # OPCION 1: Agregar un país
31 def agregar_pais(paises):
32     nombre = input("Nombre del país: ").strip().lower()
33     if not nombre:
34         print("Error: El nombre no puede estar vacío.")
35     return
36     if nombre in paises:
37         print("El país ya está en la lista.")
38     else:
39         paises.append(nombre)
40         guardar_paises(paises)
41         print("País agregado exitosamente.")
42
43 def actualizar_pais(paises):
44     nombre = input("Nombre del país a actualizar: ").strip().lower()
45     if nombre in paises:
46         poblacion = input("Población: ")
47         superficie = input("Superficie: ")
48         continente = input("Continente: ")
49         paises[nombre] = {"poblacion": poblacion, "superficie": superficie, "continente": continente}
50         guardar_paises(paises)
51         print("País actualizado exitosamente.")
52     else:
53         print("El país no está en la lista.")
54
55 def buscar_pais(paises):
56     nombre = input("Nombre del país a buscar: ").strip().lower()
57     if nombre in paises:
58         pais = paises[nombre]
59         print(f"País: {nombre}, Población: {pais['poblacion']}, Superficie: {pais['superficie']}, Continente: {pais['continente']}")
60     else:
61         print("El país no está en la lista.")
62
63 def filtrar_paises(paises):
64     continente = input("Continente a filtrar: ").strip().lower()
65     paises_filtrados = [pais for pais in paises if pais['continente'].lower() == continente]
66     guardar_paises(paises_filtrados)
67     print(f"Países filtrados por continente: {continente}")
68
69 def ordenar_paises(paises):
70     orden = input("Ordenar por: 1. Población, 2. Superficie, 3. Continente ").strip().lower()
71     if orden == "1":
72         paises_ordenados = sorted(paises, key=lambda pais: pais['poblacion'])
73     elif orden == "2":
74         paises_ordenados = sorted(paises, key=lambda pais: pais['superficie'])
75     elif orden == "3":
76         paises_ordenados = sorted(paises, key=lambda pais: pais['continente'])
77     else:
78         print("Opción no válida.")
79     guardar_paises(paises_ordenados)
80     print("Países ordenados exitosamente.")
81
82 def mostrar_estadisticas(paises):
83     print("Estadísticas de los países:")
84     total_paises = len(paises)
85     total_poblacion = sum(pais['poblacion'] for pais in paises)
86     total_superficie = sum(pais['superficie'] for pais in paises)
87     total_continente = sum(pais['continente'] for pais in paises)
88     print(f"Total de países: {total_paises}")
89     print(f"Total de población: {total_poblacion}")
90     print(f"Total de superficie: {total_superficie}")
91     print(f"Total de continentes: {total_continente}")
92
93 def salir():
94     print("¡Adiós!")
95
96 def main():
97     paises = {}
98     while True:
99         print("\n--- MENÚ ---")
100         opciones = [
101             "1. Agregar país",
102             "2. Actualizar país",
103             "3. Buscar país",
104             "4. Filtrar países",
105             "5. Ordenar países",
106             "6. Mostrar estadísticas",
107             "7. Salir"
108         ]
109         for opcion in opciones:
110             print(opcion)
111         opcion = input("Elija una opción: ")
112         if opcion == "1":
113             agregar_pais(paises)
114         elif opcion == "2":
115             actualizar_pais(paises)
116         elif opcion == "3":
117             buscar_pais(paises)
118         elif opcion == "4":
119             filtrar_paises(paises)
120         elif opcion == "5":
121             ordenar_paises(paises)
122         elif opcion == "6":
123             mostrar_estadisticas(paises)
124         elif opcion == "7":
125             salir()
126         else:
127             print("Opción no válida.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Moni-Book\Desktop\Entregable\Programación TPI>

--- MENÚ ---

1. Agregar país

2. Actualizar país

3. Buscar país

4. Filtrar países

5. Ordenar países

6. Mostrar estadísticas

7. Salir

Elija una opción: 1

Nombre del país: Chile

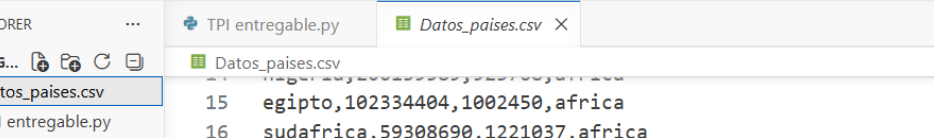
Población: 19500000

Superficie: 1756000

Continente: america

> OUTLINE

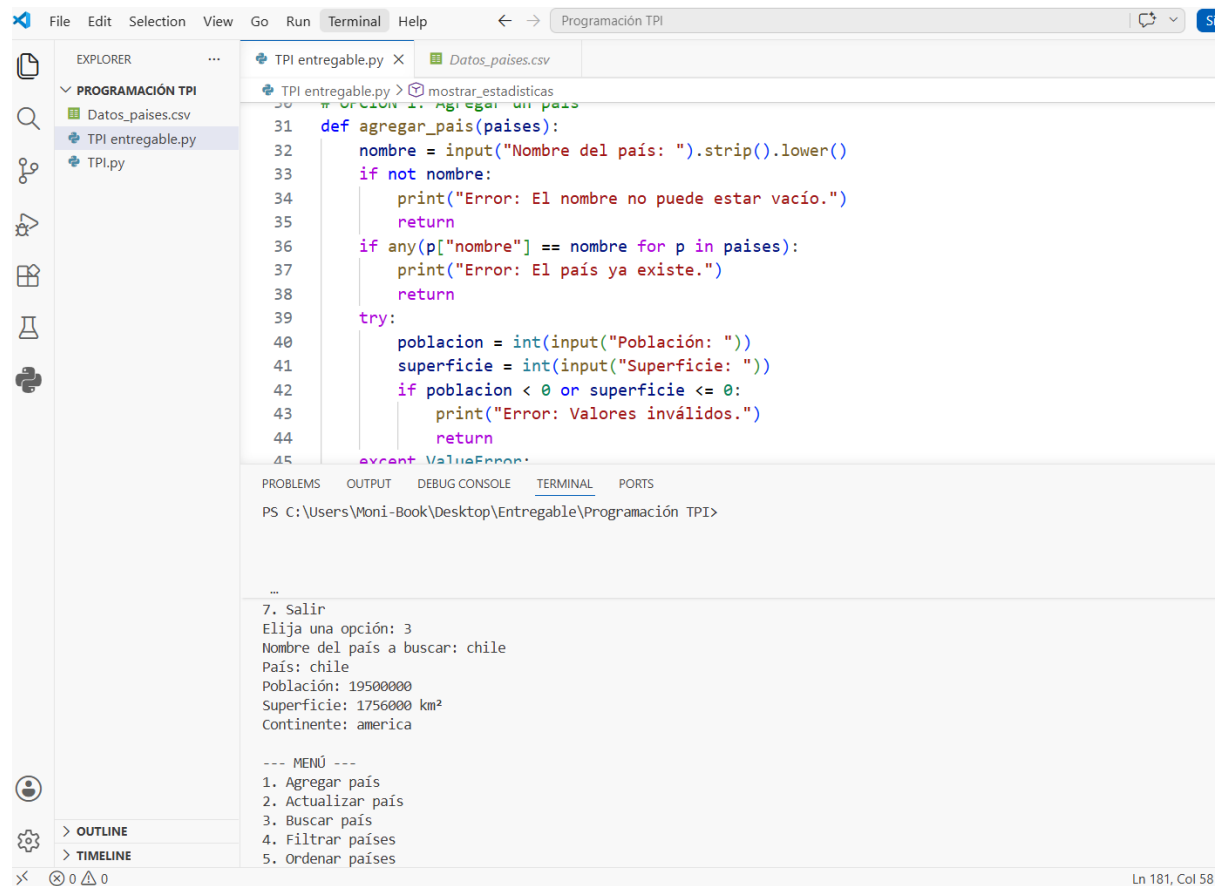
> TIMELINE



The screenshot shows a code editor with the Explorer panel on the left. The Explorer panel displays the file structure, including a folder named 'PROG...' and a file named 'Datos_paises.csv'. The file 'Datos_paises.csv' is selected, and its contents are displayed in the main editor area. The file contains a list of countries with their names, coordinates, and continents.

Line	Country	Coordinates	Continent
15	egipto	102334404,1002450	africa
16	sudafrica	59308690,1221037	africa
17	méxico	128932753,1972550	america
18	indonesia	273523615,1904569	asia
19	italia	60483973,301340	europa
20	españa	46754778,505990	europa
21	nueva zelanda	4822233,268021	oceania
22	angola	5,5	africa
23	chile	19500000,1756000	america
24			

Si lo buscamos con opción 3. muestra los datos guardados previamente.



The screenshot shows a Visual Studio Code editor window with the following components:

- Explorer Panel:** Shows a project named "PROGRAMACIÓN TPI" containing files "Datos_paises.csv", "TPI entregable.py", and "TPI.py".
- Editor Panel:** Displays the code for "TPI entregable.py". The code defines a function `agregar_pais(paises)` that takes a list of countries and adds a new one. It includes input validation for the country name, population, and area. The function returns a list of countries with their respective data.
- Terminal Panel:** Shows the output of the program. It displays the prompt "7. Salir" and the user's input "3". The program then prompts for the country name, population, and area, and displays the resulting data for "chile".

```
def agregar_pais(paises):
    nombre = input("Nombre del país: ").strip().lower()
    if not nombre:
        print("Error: El nombre no puede estar vacío.")
        return
    if any(p["nombre"] == nombre for p in paises):
        print("Error: El país ya existe.")
        return
    try:
        poblacion = int(input("Población: "))
        superficie = int(input("Superficie: "))
        if poblacion < 0 or superficie <= 0:
            print("Error: Valores inválidos.")
        return
    except ValueError:
```

7. Salir
Elija una opción: 3
Nombre del país a buscar: chile
País: chile
Población: 19500000
Superficie: 1756000 km²
Continente: america

--- MENÚ ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países

Opción 7. Salir: finaliza el programa

```
File Edit Selection View Go Run Terminal Help
TPI entregable.py X Datos_paises.csv
PROGRAMACIÓN TPI
  Datos_paises.csv
  TPI entregable.py
  TPI.py

126 def ordenar_paises(paises):
145     f"Superficie: {p['superficie']} km² | "
146     f"Continente: {p['continente']}")
147
148 # OPCIÓN 6: Estadísticas
149
150 def clave_poblacion(pais):
151     return pais["poblacion"]
152
153 def clave_superficie(pais):
154     return pais["superficie"]
155
156 def mostrar_estadisticas(paises):
157     if not paises:
158         print("No hay datos cargados.")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Moni-Book\Desktop\Entregable\Programación TPI>

...
Superficie: 1756000 km²
Continente: america

--- MENÚ ---
1. Agregar país
2. Actualizar país
3. Buscar país
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
Elija una opción: 7
Fin del programa.
PS C:\Users\Moni-Book\Desktop\Entregable\Programación TPI>
```

Dificultades y Conclusiones:

Durante el desarrollo del trabajo surgieron varias dificultades técnicas que marcaron mi proceso de aprendizaje. Una de las principales fue lograr que el programa pudiera llamar correctamente al archivo CSV, leerlo y guardar los cambios de manera persistente. También me costó implementar las funciones de ordenamiento `sort()`, ya que no las había utilizado antes, y en ese punto me apoyé en Copilot para entender mejor la lógica y estudiar la herramienta como apoyo de aprendizaje (funciones auxiliares). También tuve demoras en llamar correctamente los diccionarios y varias pruebas.

En otros proyectos de la tecnicatura, había trabajado con la librería `pandas`, por lo que tuve que practicar con las herramientas estándar de Python, como listas, diccionarios y el módulo `csv`, para cumplir con la consigna. Otro desafío fue la lectura de la documentación oficial de Python, que resultó compleja al principio y requirió tiempo para interpretar ejemplos y aplicarlos al proyecto.

En conclusión, el trabajo permitió aplicar los conceptos fundamentales de programación en Python y, al mismo tiempo, enfrentar problemas reales de implementación. Resolverlos me ayudó a comprender mejor las estructuras básicas, valorar la importancia de usar la librería estándar, reconocer la utilidad de la documentación oficial y aprender a usar Copilot no solo como generador de código, sino como herramienta para entender y mejorar mi propio razonamiento. La experiencia fortaleció mi capacidad de resolver problemas, investigar y aplicar buenas prácticas, dejando como resultado un aprendizaje sólido que me prepara para proyectos más complejos en el futuro.

Bibliografía Bibliografía / Webgrafía

<https://docs.python.org>

Anexo

Link al repositorio del proyecto

https://github.com/DaniBu56/Programaci-n_TPI

Link al video explicativo del proyecto

https://youtu.be/DcTu_b5lokg